AWS Certified Data Engineer Associate: Architecture Scenarios

A Visual Study Guide for Scenarios

2

Unofficial Study Guide based on 'sthithapragna' Exam Prep

NotebookLM

Automating S3 Partition Updates in the Data Catalog

Challenge:

New partitioned data arrives in S3 daily. Athena queries require immediate access to these new partitions without manual intervention or full crawler rescans.

Solution:

Create Partition API / Event-Driven Lambda

Explanation:

Relying on scheduled crawlers or manual MSCK REPAIR introduces latency. Using an event-driven flow programmatically registers new partitions in real-time, making them instantly queryable.

NotebookLM

Ingesting Third-Party SaaS Data into S3

Challenge: Extract data from external SaaS applications (Salesforce, Google Analytics, ServiceNow) into an S3 data lake and Redshift with minimal development and maintenance.

Solution: Amazon AppFlow

Explanation: AppFlow is a fully managed integration service designed for SaaS platforms. It handles API authentication, schema mapping, and incremental syncs automatically.

NotebookLM

Fixing Missing Aggregations in Athena Queries

Challenge:
A SQL query grouping sales data by product name is missing products that have null or zero sales, despite them existing in the S3 table.

Solution:
Use the **HAVING Clause**

Explanation:
The WHERE clause filters rows before aggregation, prematurely excluding rows with null/zero values. Applying HAVING evaluates the condition after aggregation.

Incorrect Approach

~~`WHERE sum(sales_amount) >= 0`~~

Correct Solution

```
HAVING sum(sales_amount) >= 0
```

✗ Filters pre-aggregation

✓ Filters post-aggregation

NotebookLM

Ad-Hoc Queries on Parquet Files Without Infrastructure

Challenge: Perform a one-time query to retrieve a single column from a large Parquet file in S3, avoiding the complex setup of Glue or Athena.

Solution: **Amazon S3 Select**

Explanation: S3 Select enables direct SQL querying of S3 object data. Because Parquet is a columnar format, S3 Select efficiently fetches only the required column, minimizing data transfer and compute costs.

NotebookLM

Automating Redshift Materialized View Refreshes

Challenge:

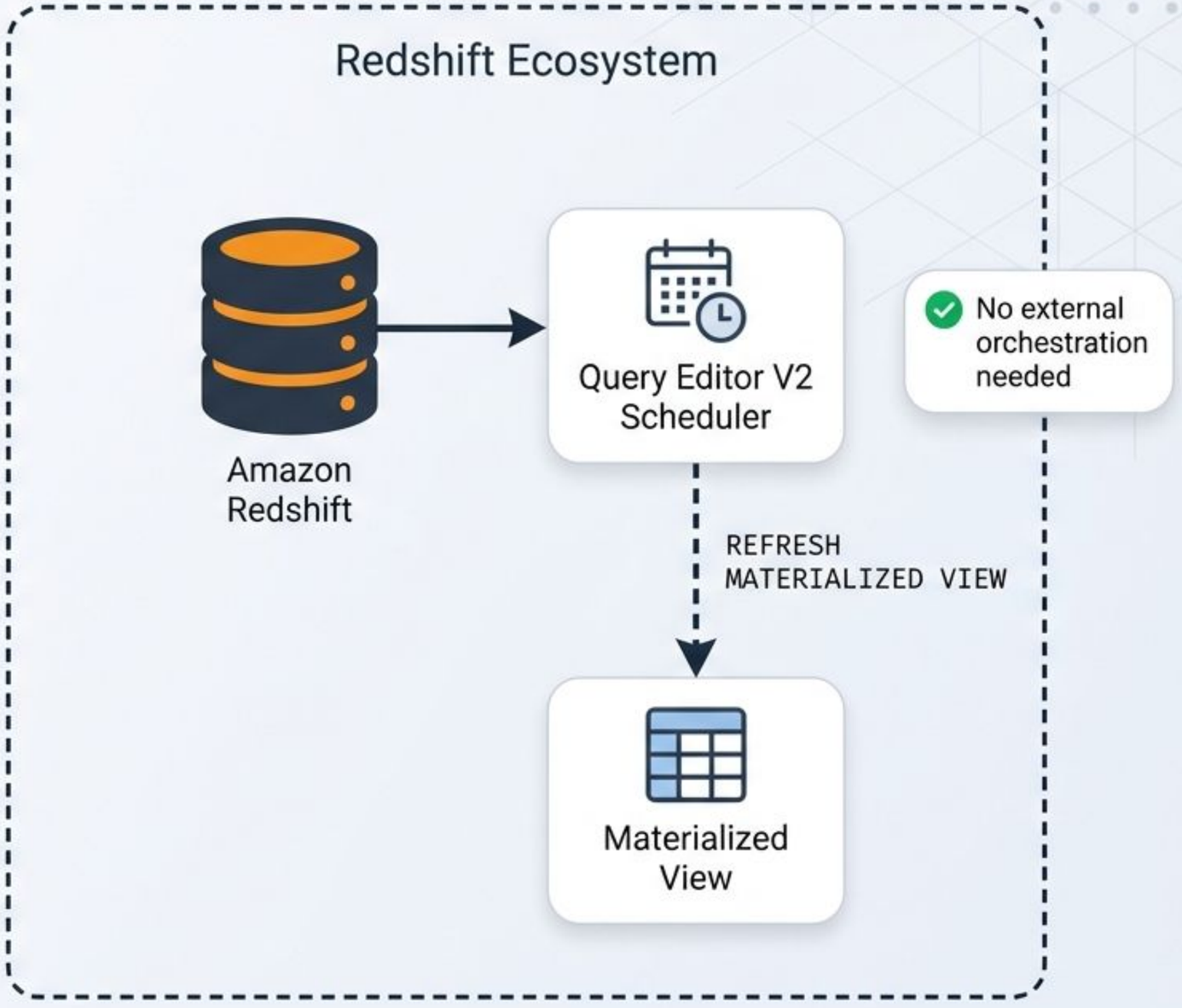
Refresh Redshift materialized views on a recurring schedule to keep analytics up-to-date, using a simple, low-maintenance approach.

Solution:

Redshift Query Editor V2

Explanation:

Query Editor V2 includes a built-in scheduling feature. Users can run SQL statements natively within the Redshift environment, requiring no external orchestration tools like EventBridge or Airflow.



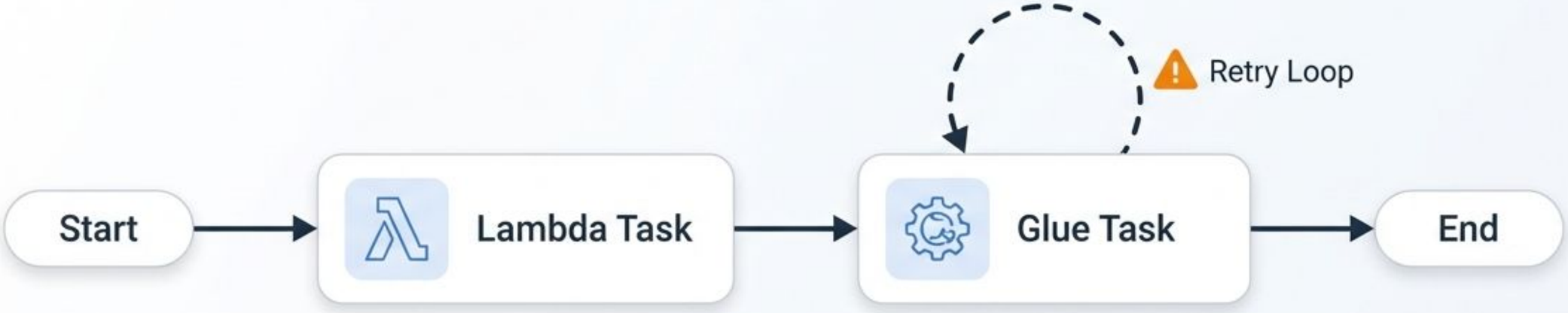
NotebookLM

Orchestrating Serverless ETL Pipelines

Challenge: Run a sequential pipeline where an AWS Lambda function executes first, followed by an AWS Glue job, requiring seamless integration and minimal management overhead.

Solution: **AWS Step Functions**

Explanation: Step Functions natively integrate with both Lambda and Glue. By defining a state machine, it handles execution order, error handling, and retries in a fully serverless environment.



NotebookLM

Centralizing Metadata Across Diverse AWS Sources

Challenge:

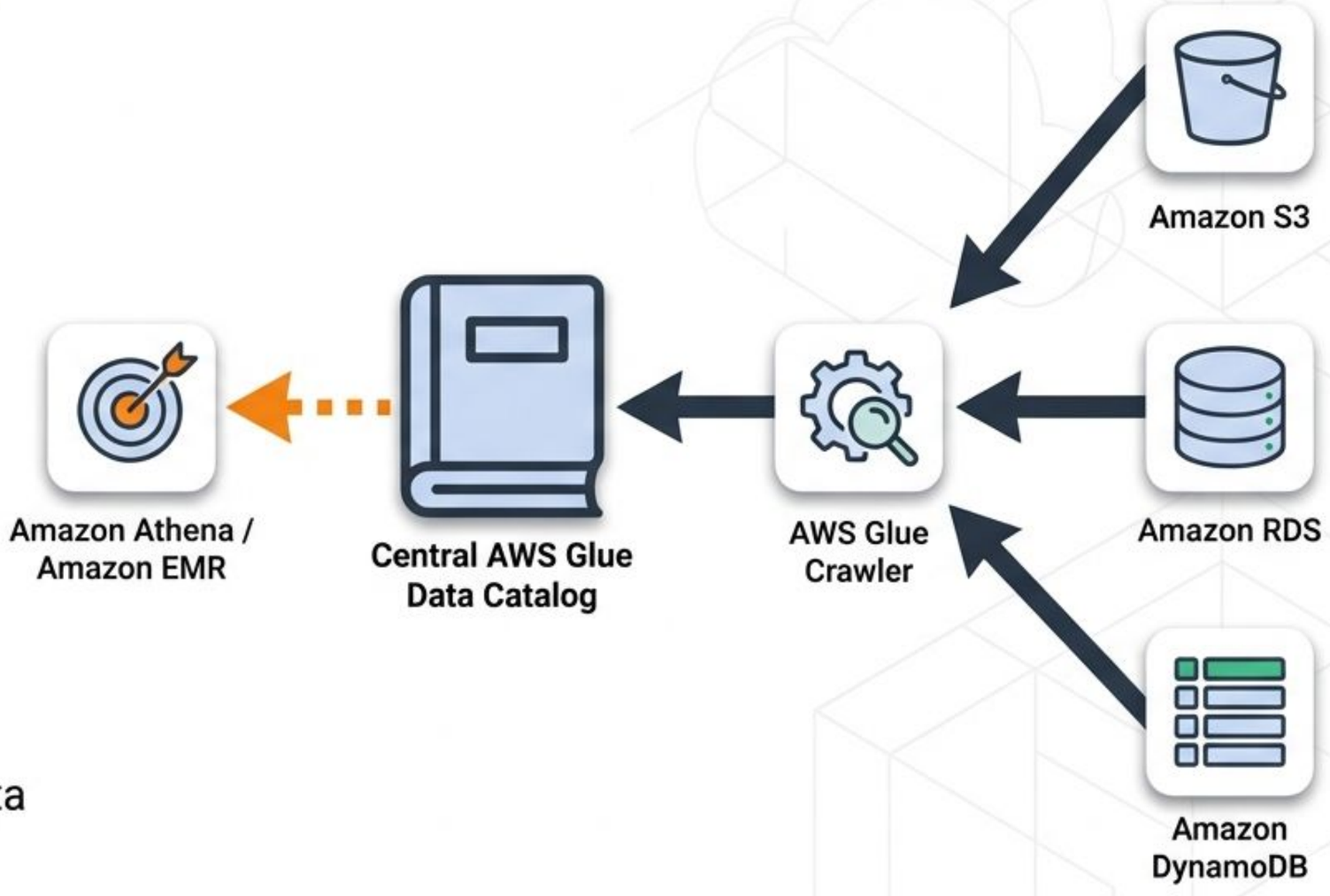
Create an automated, centralized metadata catalog for structured and semi-structured data spanning S3, RDS, DynamoDB, and Redshift.

Solution:

AWS Glue Data Catalog & Crawlers

Explanation:

Glue Crawlers automatically connect to various AWS data stores, infer evolving schemas, and populate the Data Catalog. This serves as a unified metadata repository for query engines.



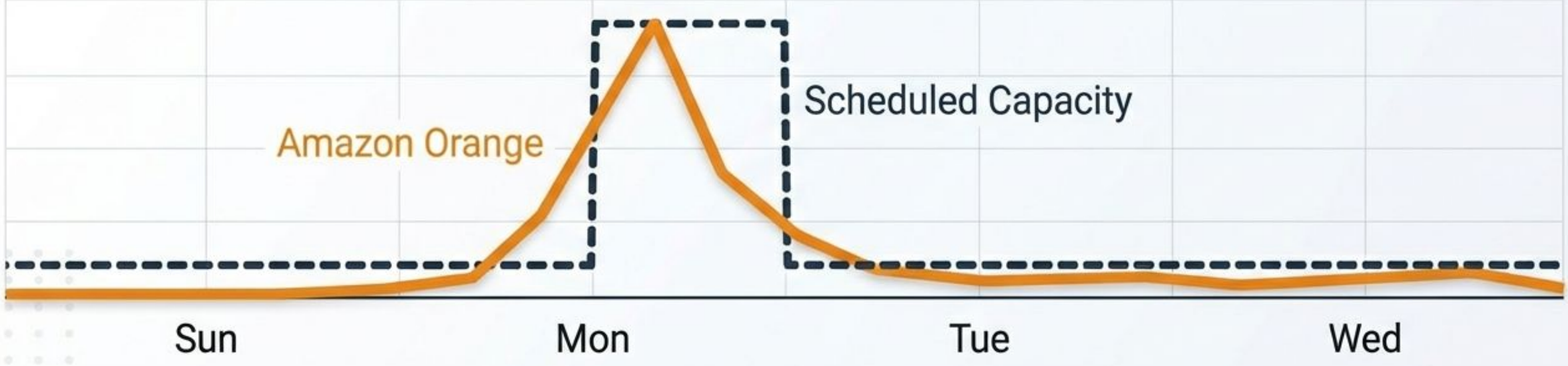
NotebookLM

Scaling DynamoDB for Predictable Workloads

Challenge: A DynamoDB table in provisioned capacity mode experiences predictable traffic spikes every Monday morning and low usage on weekends. Optimize costs without sacrificing peak performance.

Solution: Application Auto Scaling (Scheduled Scaling)

Explanation: Because the traffic pattern is known in advance, Scheduled Scaling is more cost-effective than On-Demand mode. It automatically adjusts Capacity Units (RCUs/WCUs) based on predefined time schedules.



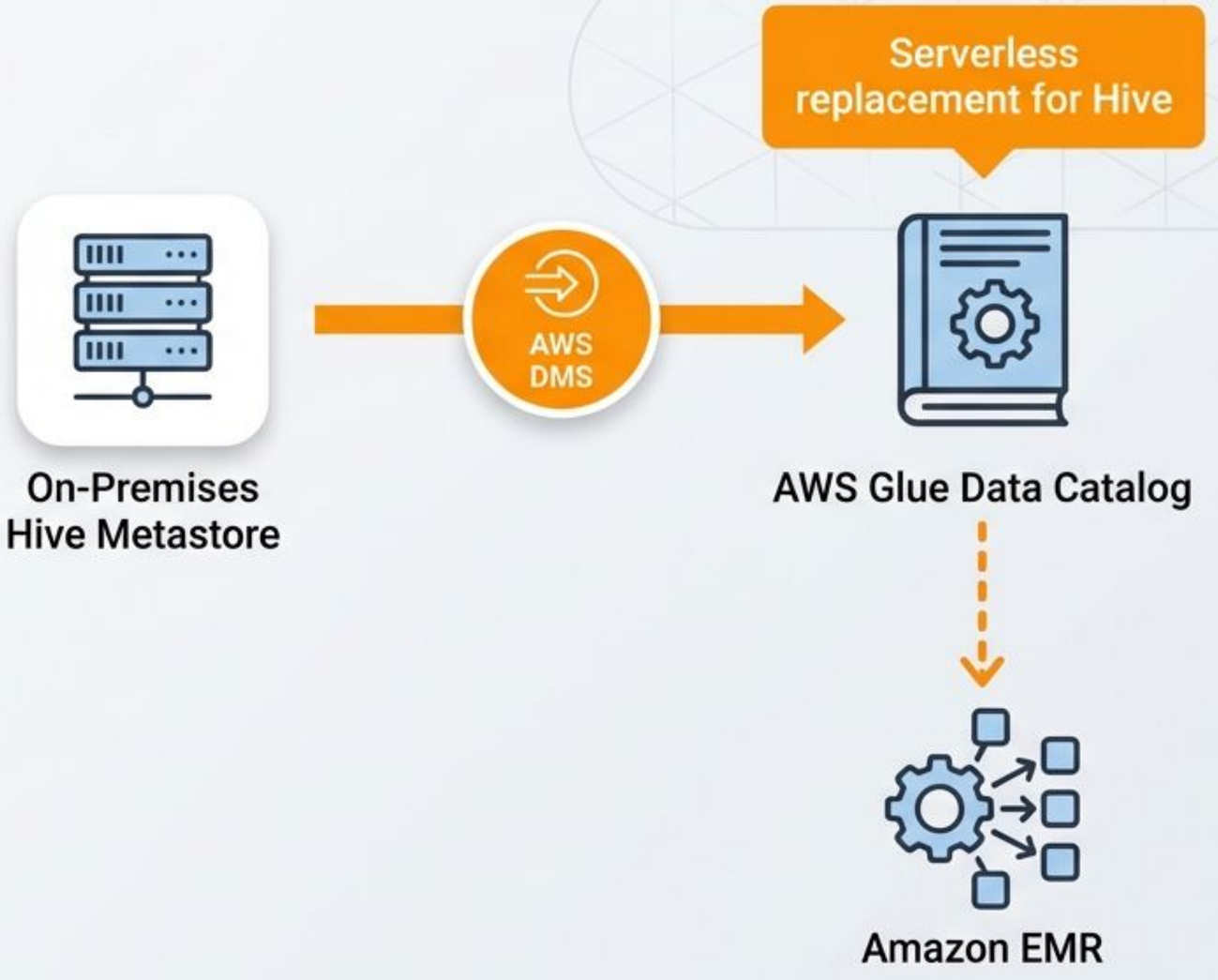
NotebookLM

Migrating On-Premises Hadoop & Hive Metastore

Challenge: Migrate on-premises Apache Hadoop workloads to AWS EMR and find a serverless, cost-effective replacement for the Hive Metastore.

Solution: AWS Glue Data Catalog (via AWS DMS)

Explanation: The Glue Data Catalog serves as a fully managed, serverless drop-in replacement for the Hive Metastore. AWS Database Migration Service (DMS) can migrate the existing metadata, retaining schema definitions seamlessly.



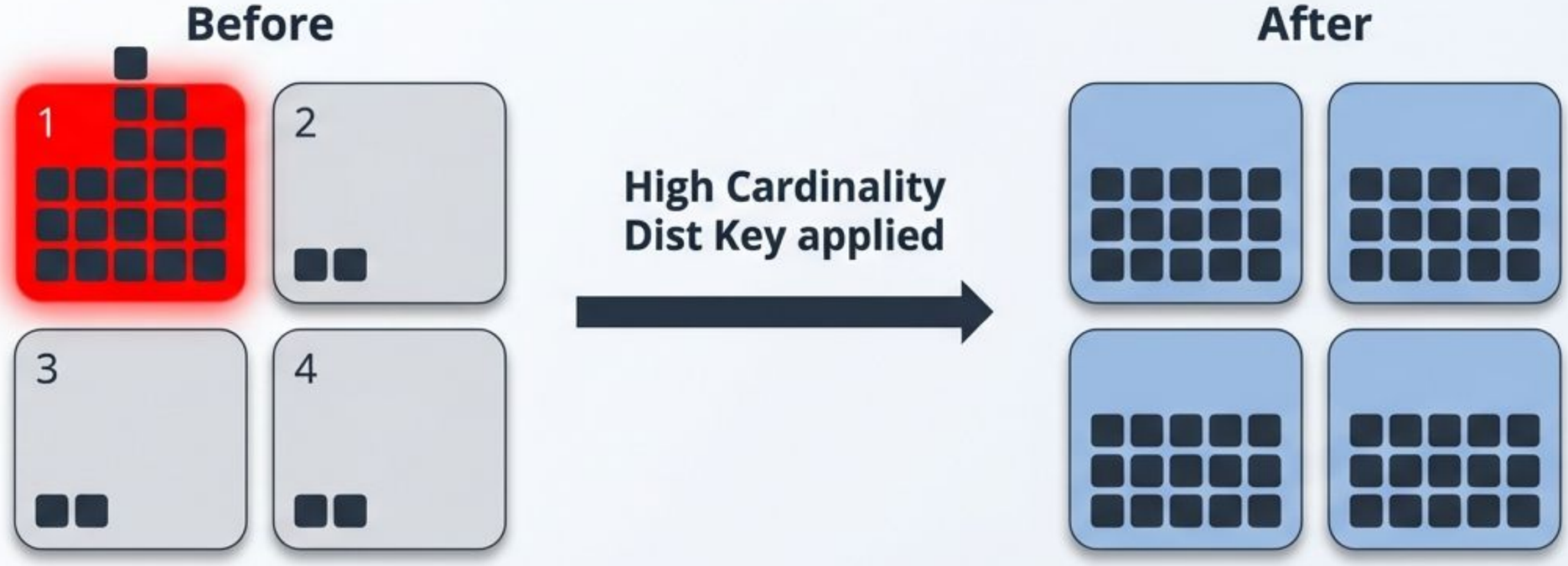
NotebookLM

Resolving Redshift Skewed CPU Utilization

Challenge: In a provisioned Redshift cluster, one node runs at >90% CPU (a hot node) while others are under 15%. Load must be balanced without increasing cluster size.

Solution: **High Cardinality Distribution Key**

Explanation: Uneven data distribution causes hot nodes and query queuing. Assigning a column with high cardinality (many unique values) as the Distribution Key ensures data is spread evenly across all compute slices.



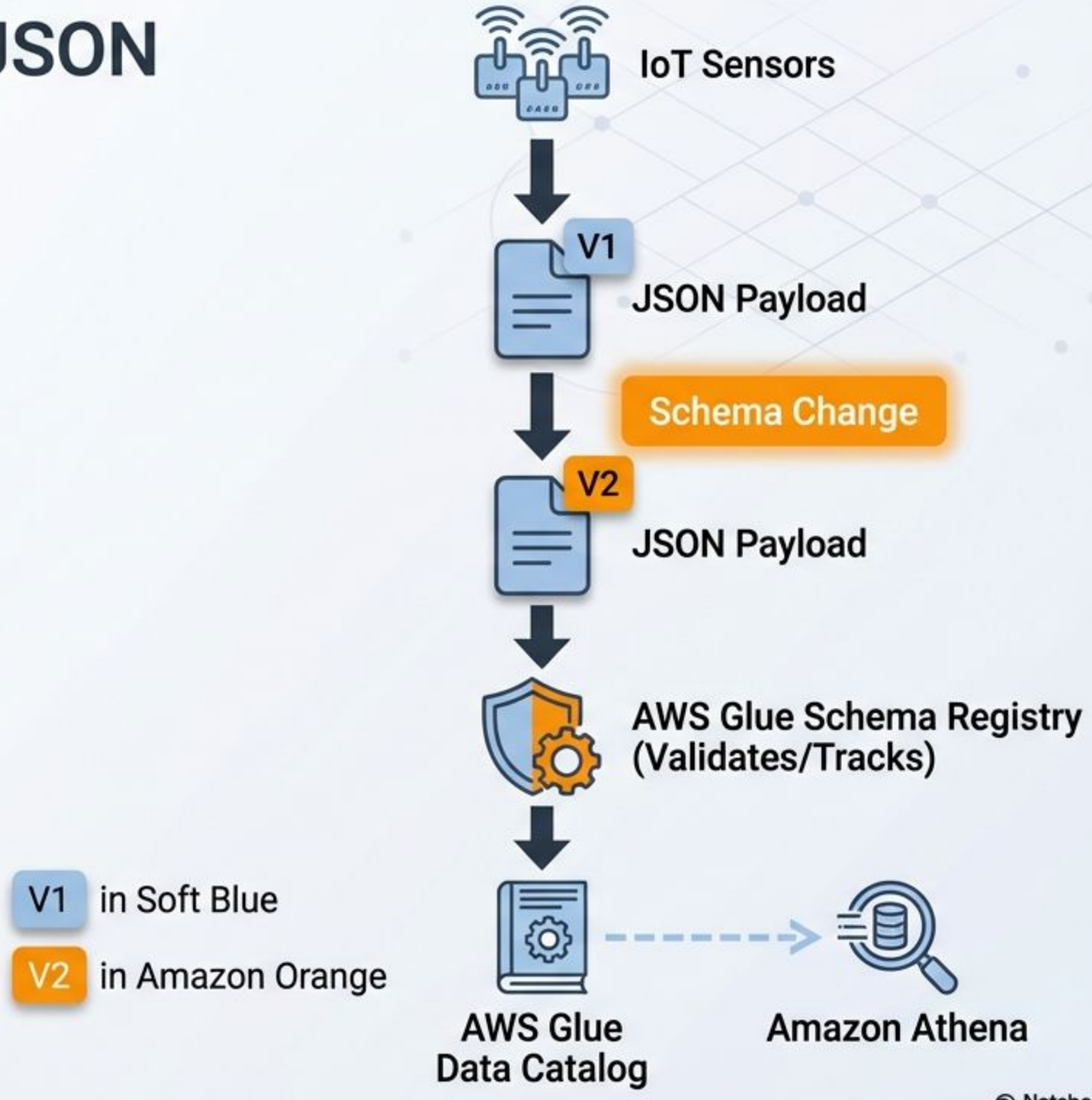
NotebookLM

Adapting to Evolving IoT JSON Schemas

Challenge: IoT devices send JSON data to S3, but the data structure frequently changes due to device upgrades. Downstream exploration needs an adaptable, cost-effective catalog.

Solution: **Glue Schema Registry + Glue Data Catalog**

Explanation: The Glue Schema Registry handles schema evolution automatically, tracking and validating structural changes in streaming data. Combined with the Data Catalog, analysts can query evolving IoT data without manual updates.



NotebookLM

Auditing S3 Write Operations

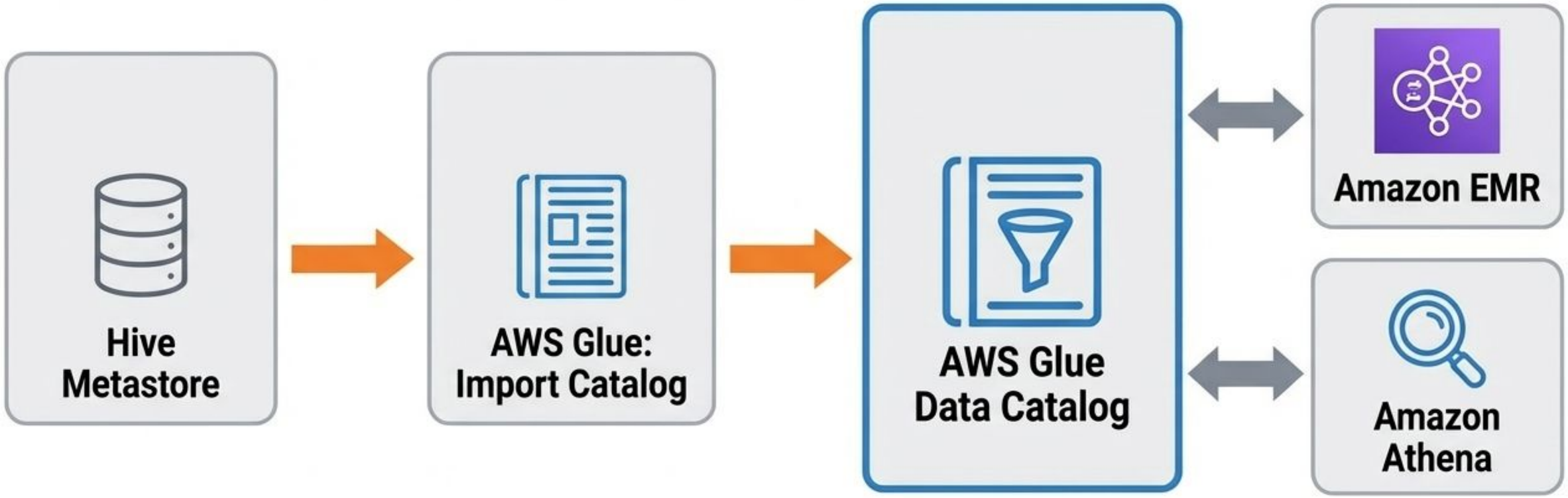
Log every write operation | Minimal setup | Automated storage

Enable CloudTrail Data Events to monitor and automatically log all object-level API operations (like PUT and DELETE) to a separate destination bucket with zero custom scripting.

NotebookLM

Centralizing Metadata from EMR & Athena

Central metadata repository | Existing Hive metastore | Minimize development effort

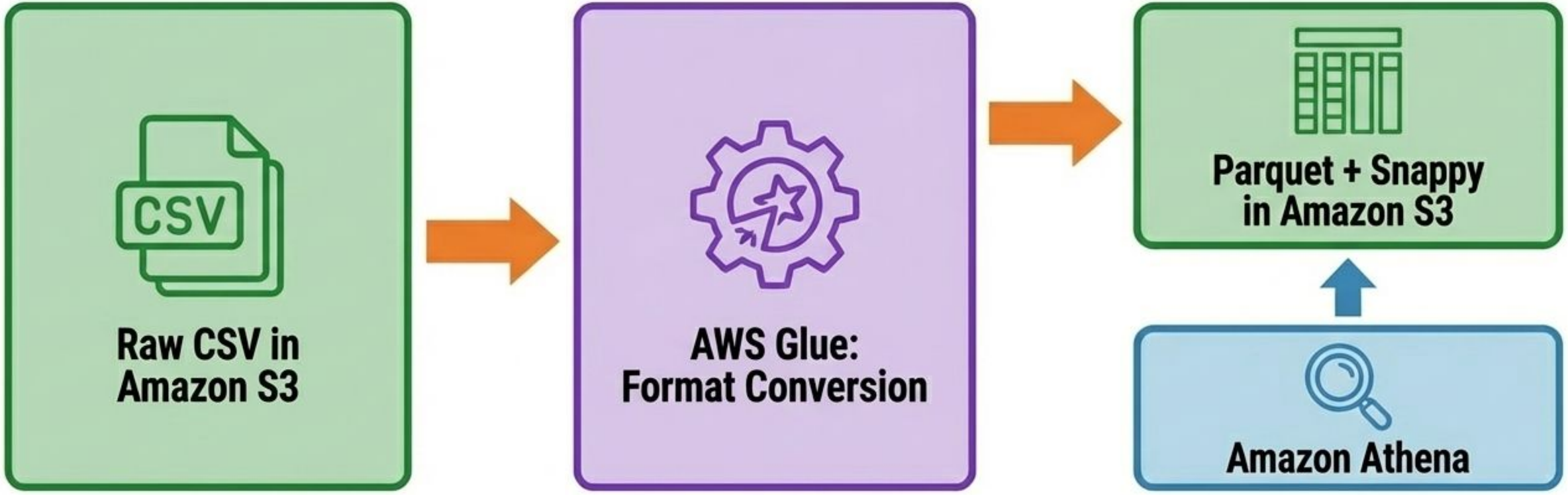


Use the Glue Import Catalog feature (or EMR Hive configuration) to migrate existing Hive metadata to the serverless Glue Data Catalog, enabling a single seamless query layer for both Athena and EMR.

NotebookLM

Optimizing Storage for Athena Analytics

CSV files | Optimize storage | Query slowdown

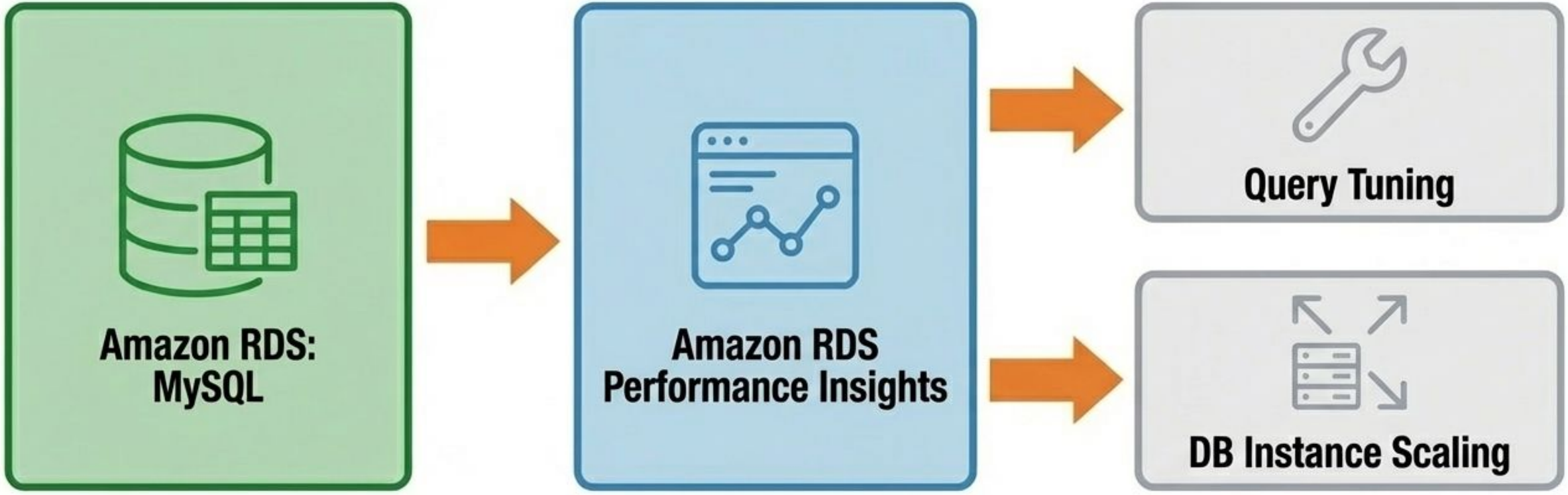


Convert uncompressed CSV files to Parquet (a columnar storage format) combined with Snappy compression. This drastically reduces Athena scan times and storage costs by reading only requested columns.

NotebookLM

Resolving RDS CPU Bottlenecks

Write-heavy RDS | CPU utilization spiked | Lower CPU usage



First, leverage Performance Insights to pinpoint and tune resource-heavy SQL statements. If software-level bottlenecks persist, scale the DB instance class hardware to increase available compute and memory.

NotebookLM

Reclaiming Space in Redshift Interleaved Sort Keys

Interleaved sort key | Frequent updates and deletes | Reclaim disk space

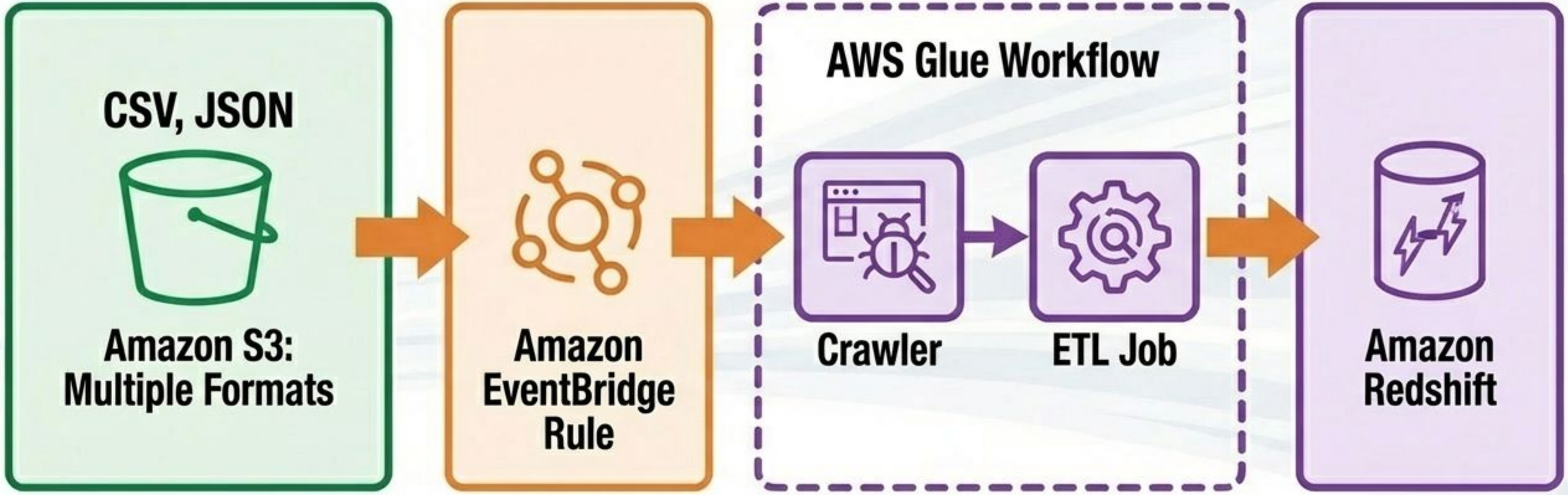
Execute the VACUUM REINDEX command specifically to rebuild the index and reclaim wasted storage space caused by fragmented interleaved sort keys after heavy delete and update operations.

NotebookLM

Handling Evolving Schemas at Scale

Handle schema changes automatically

Multiple source systems



Orchestrate an AWS Glue Workflow triggered by EventBridge. A Glue Crawler first detects dynamic schema changes and updates the Data Catalog before the Spark ETL job processes and loads the data.

NotebookLM

Reducing Athena Query Costs

Updated once daily | Refreshes every hour | Reduce Athena costs

Enable Athena's Query Result Reuse feature. Repeated queries run within the same day will instantly fetch cached results without scanning S3, drastically lowering compute costs and improving BI response times.

NotebookLM

Optimizing Redshift Queries Under Budget

Strict budget limits | Cannot be resized | Improve query performance

Amazon Redshift Workload

Distribution Keys: Node Collocation

Collocation reduces shuffles

Sort Keys: Block Pruning

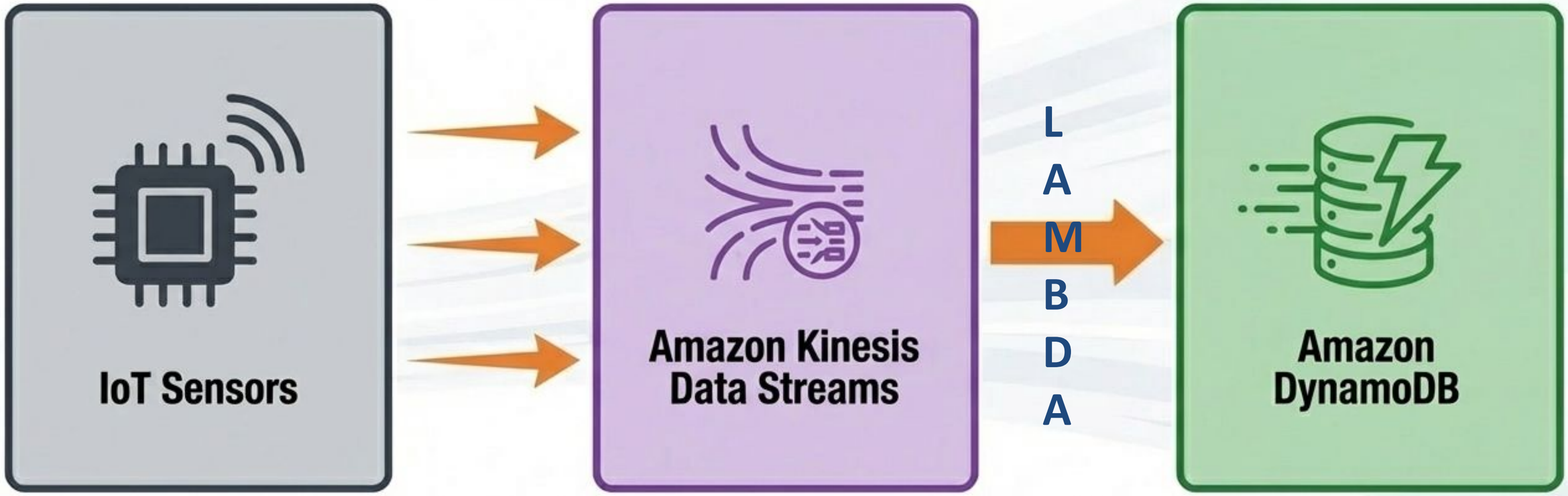
Pruning speeds up scans

Optimize the physical data model to maximize current hardware. Assign Distribution Keys to collocate heavily joined data on the same nodes (reducing shuffles) and Sort Keys to enable efficient data block pruning during scans.

NotebookLM

Near Real-Time IoT Ingestion

IoT sensor data | Nested JSON | Under 10 milliseconds latency



Stream high-throughput device data via Kinesis Data Streams + Lambda Consumer → directly into DynamoDB. DynamoDB natively supports nested JSON document storage and delivers the required single-digit millisecond read/write latency.

NotebookLM